F29AI - Artificial Intelligence and Intelligent Agents

Search and Automated Planning

Coursework 1

Nashitah Abdia Shahid
H00555549

Tanvi Sandeep Diwakar
H00561261

Canvas group / campus
TO COMPLETE: campus and Canvas group

Video demonstration
TO COMPLETE: video-demo URL

Use of Generative AI
OpenAI Codex was used to analyse the supplied coursework briefs, generate the Sudoku solver and PDDL models, develop and run tests and benchmarks, and draft and format this report. It also assisted with source lookup and preparation of the demonstration outline.

Report structure
Part 1: CSP modelling, implementation, evaluation and theoretical A* analysis (four pages). Part 2: PDDL modelling, missions and astronaut extension (three pages). References follow separately.

27 October 2026

PART 1A / F29AI COURSEWORK 1

Sudoku as a constraint problem

Variables and domains

Let X(r,c) represent the digit at row r and column c, for r,c in {1,...,9}. There are 81 variables. A blank cell initially has domain {1,...,9}; a clue of value v has singleton domain {v}. The supplied clues are fixed. A solution assigns a value to every variable while satisfying all constraints. Zero is an input marker for an unassigned cell, not a legal solution value.

Constraints

Use 27 all-different constraints: one over each of the nine rows, one over each of the nine columns, and one over each 3x3 box. Formally, AllDifferent(X(r,1),...,X(r,9)) holds for every r, and AllDifferent(X(1,c),...,X(9,c)) for every c. For a,b in {0,1,2}, the box constraint covers {X(3a+i,3b+j): i,j in {1,2,3}}. Because nine distinct values are drawn from {1,...,9}, each unit contains every digit exactly once.

A small constraint example

If the first row is [0,2,3,4,5,6,7,8,9], its missing value can only be 1. The column and box must also permit 1. If a peer already contains 1, the blank has no legal value and the branch is impossible. The implementation represents the row, column and box rules as inequalities between peer cells; each cell has 20 distinct peers.

Brute force versus backtracking

Let m be the number of initially blank cells. Brute force can enumerate 9^m complete assignments before testing the constraints, giving O(9^m) candidates. Checking a whole fixed-size grid is bounded work; if counted explicitly, a simple validator costs O(81) per candidate. Plain backtracking assigns a cell and rejects a partial assignment as soon as a row, column or box conflicts. Its loose search-tree bound is 1+9+...+9^m = (9^(m+1)-1)/8, so the worst-case upper bound remains exponential.

Why pruning matters in practice

A conflict at depth d can discard all 9^(m-d) raw completions below that assignment. Forward checking detects empty future domains; propagating forced values can remove multiple search levels. MRV selects a constrained cell to expose failures early. These techniques reduce explored work but do not guarantee polynomial time. For a fixed 9x9 board the entire input family is finite: the exponential discussion is a search-space bound parameterised by m, not an asymptotic claim about an unbounded fixed grid. Generalising the board size gives the usual growing CSP setting.

Solver design and correctness

Chosen approach

The implemented method is depth-first backtracking with minimum remaining values (MRV) and singleton constraint propagation. This combines explicit exhaustive search with sound elimination of impossible candidates. Norvig discusses the general combination of propagation and search [1]; the submitted implementation uses integer bit masks, a queue and copied branch state. It imports no Sudoku, CSP or constraint-solver library. Hidden singles, pair rules and A* are not implemented.

Implementation sequence

solver.py validates the input, creates 81 nine-bit candidate masks, and queues all givens. Each singleton removes its value from its peers; every new singleton is queued. An empty domain rejects the branch immediately. At a propagation fixed point, a grid of singletons is complete. Otherwise the search chooses the smallest non-singleton domain, breaks ties by row-major index, and tries digits in ascending order. Each attempt copies the domain list, propagates and recurses; failure discards the copy.

Search outline

```
search(domains):
    if all domains are singleton: return domains
    cell = smallest non-singleton domain (MRV)
    for digit in cell candidates, ascending:
        child = copy(domains); child[cell] = {digit}
        if propagate(child) succeeds:
            result = search(child)
            if result exists: return result
    return failure
```

Soundness, completeness and storage

Propagation removes only values forbidden by an assigned peer, so it preserves every possible completion of the branch. Branching tries every remaining value of the selected cell. With finite domains, every recursive branch assigns at least one previously non-singleton cell; exhaustive failure therefore means no solution exists. At a solved leaf, all peers differ and all givens remain fixed. Independent tests check those conditions directly. Depth is at most m; copying 81 masks at each depth uses O(81m) branch storage, excluding fixed peer data. Worst-case search remains exponential.

Input, output and trace example

The CLI accepts nine rows of nine digits in .txt or .csv files, with 0 for blanks. It reports malformed input and duplicate clues separately from a consistent but unsatisfiable puzzle. Solved grids include box separators; --stats displays counters. In Norvig entry 1, initial propagation leaves the first MRV decision at row 7, column 2 with candidates {8,9}; 8 is tried first. The solver returns one solution and does not certify uniqueness.

Measured evaluation

Protocol and dataset

Nine cases were evaluated on 27 October 2026 using Python 3.12.14 on an Apple M4, macOS 27.0.1. Each had one warm-up and seven measured solves. The table reports median wall-clock time from perf_counter, including validation, initialisation, search and counters, but excluding file I/O, process startup and output checks. All runs used the same deterministic ordering. Norvig 1-5 are the first five top95 entries [2]; the other cases are documented constructions, without assumed human difficulty ratings.

Results

Case	Givens	Median ms	Guesses	Backtracks	Result
Nearly complete	72	0.050	0	0	Solved
Generated-45	36	0.078	0	0	Solved
Norvig 1	17	0.667	130	119	Solved
Norvig 2	17	2.311	452	441	Solved
Norvig 3	17	2.294	477	463	Solved
Norvig 4	17	116.664	24,686	24,668	Solved
Norvig 5	17	29.422	6,188	6,175	Solved
Unsat-immediate	9	0.019	0	0	No solution
Unsat-search	18	0.066	6	6	No solution

What the measurements show

The constructed cases needed no guesses: propagation completed them. Norvig 4 required 24,686 guesses and 24,668 backtracks, compared with 130 and 119 for Norvig 1, despite similarly sparse clues. The hardest measured median was 116.664 ms. The immediate unsatisfiable case was rejected during propagation; the second exhausted six guesses. This shows that clue arrangement and failed branches matter, rather than clue count alone.

Definitions and test evidence

A guess is an attempted candidate assignment. A backtrack is a candidate attempt whose whole branch fails; failed parent and child attempts can both count. Counts include unsuccessful branches. All 33 unit-test methods passed, including five published-puzzle subtests and 20 generated transformations. Every returned answer was independently checked for all rows, columns, boxes and unchanged clues. Tests also cover malformed input, CSV, solved/empty grids, direct conflicts, no-solution cases and CLI exit codes. Raw repetitions, counters, file hashes and outputs are supplied in Sudoku/evaluation/.

A* analysis and limitations

An A* formulation

A state is a partial grid consistent with the clues and local Sudoku constraints. The start state is the input puzzle; a goal is a complete valid grid. An action assigns one legal digit to one selected blank cell, with cost 1. Selecting cells deterministically, for example by MRV, removes redundant assignment orders while retaining all possible completions. The path cost g is the number of assignments made. This theoretical model counts each filled cell separately; it does not treat a propagation cascade as one unit-cost action.

Heuristic, admissibility and consistency

Choose h(s) as the number b of remaining blanks. Any successful continuation needs exactly b assignments, so h equals the remaining cost for completable states; for dead ends the true remaining cost is infinite. Thus h never overestimates. A legal transition fills one cell, so h(s) = 1+h(s'), satisfying consistency. A* orders its frontier by f=g+h; the general role of admissible estimates and tie-breaking is described in [3]. If propagation were bundled into a single unit-cost action, this blank-count admissibility argument would no longer hold.

Why this heuristic gives little guidance

If the initial puzzle has m blanks, after k assignments g=k and h=m-k, hence f=m at every reachable state. All solutions also cost m. Although h is exact on completable states, it cannot distinguish promising partial grids from locally consistent dead ends. All f values tie, making the tie-breaking policy decisive; FIFO ties can behave breadth-first, while depth-preferring ties behave differently. A* is therefore not automatically equivalent to BFS in this formulation. It may retain a large frontier without providing a useful shortest-solution advantage.

Comparison and possible improvements

MRV backtracking directly addresses feasibility, rejects contradictions and retains only the current recursion path and sibling choices. A* adds frontier management while its usual optimal-path objective is uninformative here. Plain backtracking would avoid propagation overhead but may explore more failed assignments. Hidden-single or stronger all-different reasoning could prune further; least-constraining-value ordering could change which solution branch is reached first. These are possible improvements, not measured speedups: only the chosen method was implemented and timed.

Limits of the evidence

The sample contains seven solvable cases and two unsatisfiable cases, not the full top95 collection or a representative difficulty survey. Very short timings are sensitive to system noise; raw repetitions and ranges are retained. No baseline comparison, uniqueness test or performance guarantee is claimed. Completeness follows from exhaustive finite branching with sound propagation; the passing tests support the implementation but are not a proof that the code is bug-free.

Reusable planning model

Scope and representation

The implementation completes Parts 2A-2C using typed STRIPS. The base domain, lunar, has the types rover, lander and waypoint. The extension, lunar-extended, adds astronaut. Both use only :strips and :typing. The five starter filenames and all domain/problem names are preserved. Actions are parameterised; no mission-specific rover, lander or waypoint is hard-coded in either domain. The model follows the supplied coursework specification [4].

Locations, ownership and deployment

Static predicates belongs-to, lander-at and connected describe rover ownership, fixed landing sites and directed edges. No action changes these facts. An undeployed rover has aboard(r,l) and no rover-at fact. Deployment requires its owning lander at the selected waypoint, removes aboard and adds rover-at. Movement requires the directed connection and replaces the old location with the new one. Thus an undeployed rover cannot move, acquire data or collect a sample.

Data memory and successful delivery

memory-free is a shared availability token. capture-image and perform-scan both consume it and create carrying-image or carrying-scan, labelled with the acquisition waypoint. This enforces one unsent item across both data kinds. The corresponding transmit action requires that item and the rover's own lander, records image-saved or scan-saved at that lander, removes the carried item and restores memory-free. Acquisition alone never satisfies a saved-data goal. Transmission has no rover-location or prior-visit requirement, so it remains wireless.

Sample carrying and lander storage

sample-slot-free is separate from memory-free, allowing a rover to carry one sample and one data item simultaneously. collect-sample requires the rover at the source waypoint and consumes its sample slot. store-sample requires the carried sample, ownership, rover/lander co-location and lander-storage-free. It records sample-stored at the destination lander, frees the rover slot and permanently consumes the lander's storage token. The source waypoint and delivery waypoint are different parameters: a sample can be acquired elsewhere and returned home.

Model boundaries and invariants

Images, scans and samples are distinguished by kind and/or origin waypoint. No site-depletion rule is added because the brief specifies no limit on material available at a waypoint. Valid instances initialise exactly one owner per rover, one landing site per lander, one rover location or aboard status, and consistent free-slot facts. The actions preserve these properties; PDDL does not reject an arbitrarily inconsistent initial state. The base domain contains eight action schemas: deploy, move, capture-image, perform-scan, transmit-image, transmit-scan, collect-sample and store-sample.

Mission setup and astronaut extension

Landing choices and exact maps

Landing is fixed in the initial state. Mission 1 places L1 at WP1 with R1 aboard. Missions 2 and 3 place L1 and already-deployed R1 at the specified WP2; L2 is chosen to land at WP1, with R2 aboard. The unspecified WP1 choices are permitted by the brief. All rovers start with empty data and sample slots, and all landers have free sample storage.

Directed connectivity

Figure 2 is encoded as WP1 to WP2, WP1 to WP4, WP2 to WP3, WP4 to WP3, WP3 to WP5, and WP5 to WP1. Figure 3 has bidirectional pairs WP1/WP2, WP2/WP4 and WP3/WP5, plus one-way edges WP2 to WP3, WP5 to WP6 and WP6 to WP4. No reverse edges are inferred. Missions 2 and 3 use identical map facts.

Mission objectives

Mission 1 requires an image from WP5 and a scan from WP3 saved to L1, and a sample from WP1 stored in L1. Missions 2 and 3 require L1 to receive the WP3 image, WP6 scan and WP5 sample, and L2 to receive the WP2 image, WP4 scan and WP1 sample. Goals use only saved/stored predicates. Because every transmission and storage action checks ownership, each named lander determines the rover that must achieve its goals.

Part 2C: meaningful astronaut constraints

astronaut-belongs-to fixes each astronaut's lander. in-docking-bay and in-control-room represent the two internal areas without introducing separate room objects. Alice belongs to L1 and starts in its docking bay; Bob belongs to L2 and starts in its control room. go-to-control-room and go-to-docking-bay require ownership and the source room, remove that room fact and add the destination. The same lander parameter is used throughout, preventing movement between landers.

How the extension changes applicability

Deployment and sample storage additionally require an astronaut belonging to the destination lander in its docking bay. Both transmission actions additionally require that astronaut in the control room. These are action preconditions, not descriptive flags. Rover movement and acquisition remain autonomous. In the generated Mission 3 plan, Bob first moves to the docking bay to enable R2 deployment and sample storage, then returns to the control room for data. Alice moves to the control room for R1's transmissions and returns to the docking bay before its sample is stored.

Testing and observed results

Planner and procedure

The five files were parsed and all three missions solved locally using Pyperplan 2.1 under Python 3.12. Greedy best-first search used the FF heuristic: pyperplan -H hff -s gbf DOMAIN PROBLEM. The recorded runs used PYTHONHASHSEED=0 for reproducibility. This is Pyperplan's FF heuristic, not an execution of the separate FF planner. Greedy search does not establish plan optimality. Planner documentation and version-specific help informed the commands [5].

Mission results

Mission	Actions	Delivery goals	Full replay
1	10	3 of 3	Pass
2	20	6 of 6	Pass
3	24	6 of 6	Pass

Plan interpretation

Mission 1 stores the WP1 sample locally, travels WP1-WP2-WP3-WP5, and transmits the required data. In Mission 2, R2 stores the WP1 sample before visiting WP2 and WP4 for data. R1 follows WP2-WP3-WP5-WP6-WP4-WP2, carrying the WP5 sample back to L1. Mission 3 achieves the same deliveries with four astronaut room changes. Rovers need not end at their landers once their required physical deliveries are complete.

Independent replay and adverse-action checks

A separate standard-library Python checker replays every recorded action against the full PDDL preconditions and effects, verifies every goal and checks state invariants after every step. Its 43 checks pass. These include exact map transcription; no mission-specific constants in the domains; blocked actions before deployment; no reverse-edge shortcut; no transmission without the correct acquired item; shared data capacity; independent sample capacity; no remote or wrong-lander sample storage; and no second sample stored in a full lander. A modified initial state also confirms that wireless transmission works before a rover has ever visited its own lander.

Astronaut checks and verification limits

The checker rejects deployment with Bob in the control room, transmission with Alice in the docking bay, sample storage with Bob in the control room, use of another lander's astronaut, and cross-lander astronaut movement. Correct room changes enable the corresponding actions. The logs, plans, checker and results are included in validation/. VAL was not installed or run; the local checker provides focused full-model replay and behavioural checks, not a general PDDL validator or formal proof. Tests were not run with FF, Fast Downward or the online editor.

References

[1] Peter Norvig. Solving Every Sudoku Puzzle (2006).

https://norvig.com/sudoku.html

Conceptual reference for constraint propagation and search; accessed 26 October 2026.

[2] Peter Norvig. top95 Sudoku benchmark dataset.

https://norvig.com/top95.txt

Entries 1-5 used in the evaluation; periods converted to zeros. Retrieved 26 October 2026.

[3] David L. Poole and Alan K. Mackworth. Artificial Intelligence: Foundations of Computational Agents, 2nd ed., Section 3.6.1, A* Search.

https://www.cs.ubc.ca/~poole/aibook/2e/html2e/ArtInt2e.Ch3.S6.SS1.html

Reference for A* evaluation and admissibility; accessed 26 October 2026. The Sudoku-specific f=m derivation is the analysis of the formulation used here.

[4] F29AI Coursework 1: Search and Automated Planning. Supplied course brief excerpts.

task soduko.pdf: printed pages 1-2. task.pdf: printed pages 3-10. Requirements for Parts 1A/1B, 2A/2B/2C and submission format.

[5] Pyperplan project. README and documentation; Pyperplan 2.1 command help.

https://github.com/aibasel/pyperplan

Planner setup and usage; accessed 26 October 2026. The installed version uses the search option gbf.